Java Web Application Security Case Study

Kyle DeJong | Artemis Financial academic scenario | Original work April 19, 2026

This report documents a focused security refactor of a supplied Spring application for SNHU CS-305. I added a SHA-256 checksum route, generated a self-signed certificate and keystore, configured local HTTPS, and reviewed the result with manual testing and OWASP Dependency-Check. AES-256 was a recommendation, not a demonstrated implementation. The dependency scan retained serious findings, so the result is a local learning exercise rather than a production-ready application.

Artemis Financial is a fictional client in the supplied course scenario. This portfolio contains the report and historical screenshots. The application source, build package, certificates, and private keystore are not included, and the application has not been rerun for this edition.

Cryptographic Choices

As Artemis Financial deals with sensitive financial information through a web interface, I recommended AES-256 for confidentiality. AES uses 128-bit blocks and supports 128, 192, and 256-bit keys under NIST FIPS 197 [1]. A 256-bit key provides a large brute-force search space, but key size alone does not establish the security of a complete system. The mode of operation, key handling, and protocol configuration also matter. The retained evidence does not establish which cipher suite the local HTTPS connection negotiated.

For the checksum demonstration, I used SHA-256 through Java's MessageDigest functionality [2]. SHA-256 produces a fixed-length 256-bit digest rather than reversible encrypted data. The route returned a fixed test string and its digest. This can demonstrate repeatable hashing and comparison, but a plain digest delivered alongside the data does not authenticate the sender or prevent an attacker from changing both values. The original report's broader claims about tamper protection have been narrowed accordingly.

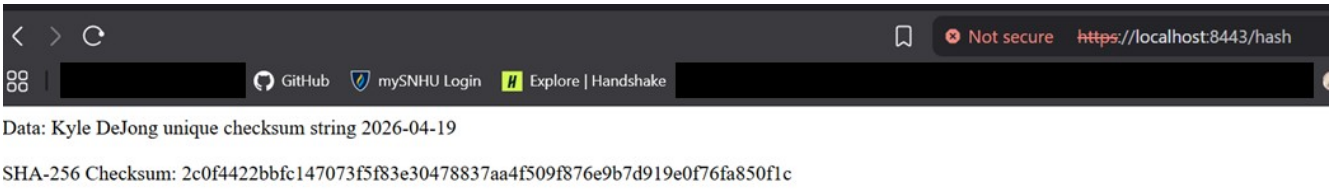
Encryption and hashing solve different problems. AES is symmetric, using the same secret key for encryption and decryption, while asymmetric cryptography uses a public and private key pair. Reliable key generation and protection are part of the surrounding design. Configuring HTTPS uses a transport security protocol rather than simply selecting a standalone cipher; certificate validation remains essential to establishing the peer's identity [4].

Certificate and HTTPS Configuration

I used Java certificate tooling to generate a self-signed certificate and keystore, then updated application.properties for HTTPS [3]. The original project-tree screenshot shows certificate.cer and keystore.p12. Their filenames support the account of the project structure, but do not independently establish the files' contents, private-key protection, or certificate trust. The screenshot is retained separately for inspection.

The application ran at https://localhost:8443/hash. The browser displayed a certificate warning because the local setup was not trusted. I attempted to add the certificate to the local machine certificate store, but the warning remained. This was accepted for the course exercise. The screenshot preserves that warning and should not be read as evidence of publicly trusted TLS.

Checksum Output and Functional Testing



Historical browser output from April 19, 2026. The original image already contains blacked-out browser areas; its bytes are preserved. The visible certificate warning remains part of the evidence.

Manual testing included running the refactored application and reviewing the added code for syntax, logic, and security concerns. I recorded that the route compiled, returned the expected original string, and produced consistent checksum output. The reported operation was read-only, used a fixed string, and added no third-party libraries. Those constraints limited the scope of the change, but successful startup and a narrow route do not prove the absence of vulnerabilities.

During portfolio cleanup, an independent SHA-256 calculation of the exact visible string reproduced the displayed digest. This checks the screenshot's input/output pair only. It is not a fresh execution of the original Java application or a validation of its TLS configuration.

Process and Reflection

My process was layered. First, I separated confidentiality from hashing, then generated the certificate and keystore, configured HTTPS, added the checksum route, and tested the result manually and with Dependency-Check. I relied on established Java tools and algorithms rather than inventing custom cryptography, and kept the refactor narrow instead of adding unnecessary libraries.

It was helpful to distinguish existing dependency issues from the changes I made. That separation defines the work's scope, but does not make inherited vulnerabilities irrelevant. The original report attributed the findings to older starter dependencies. Without a before-and-after baseline and code review, the retained scan cannot establish that my changes introduced no security issues. A usable follow-up would review the findings, update affected components, and rerun both functional and security checks. Those steps are not claimed as completed here.

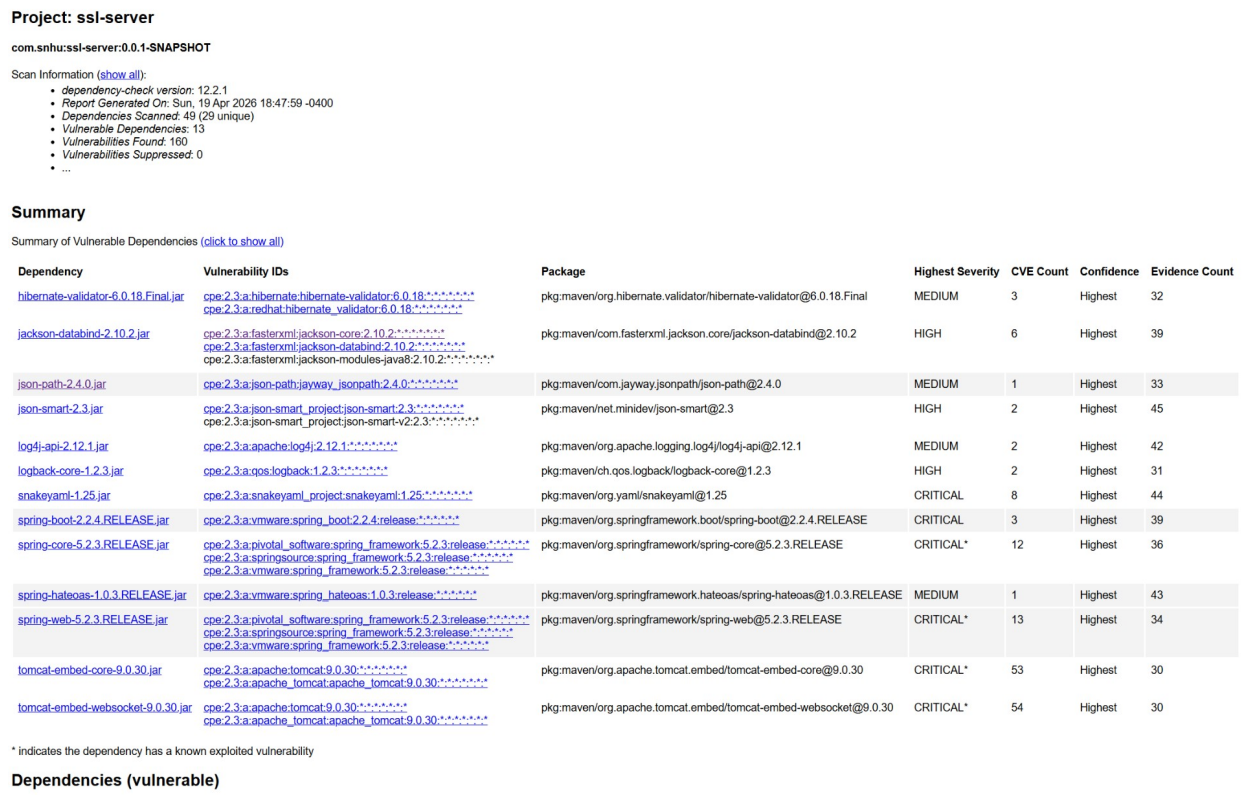
The project's value is working within an existing application and explaining a bounded change. Designing with security in mind before coding begins, during development, and after release helps avoid costly rework. Knowing where a change begins and ends helps focus effort, while documenting unresolved risks makes the result easier for the next developer to assess.

Portfolio Edition

AI-assisted cleanup removed template material, logos, and console screenshots exposing local machine details, corrected claims, and added references. Retained screenshots are unchanged. The original school report is preserved separately.

Historical Dependency Findings

OWASP Dependency-Check identifies dependencies associated with publicly disclosed vulnerabilities [5]. The retained April 19, 2026 report used version 12.2.1 and recorded 49 dependencies scanned, 29 unique dependencies, 13 vulnerable dependencies, and 160 findings. High and critical entries remain visible. These are historical scanner results, not a verified count of exploitable defects or a statement of current dependency safety.



Project: ssl-server

com.snhu:ssl-server:0.0.1-SNAPSHOT

Scan Information (show all):

- *dependency-check version*: 12.2.1
- *Report Generated On*: Sun, 19 Apr 2026 18:47:59 -0400
- *Dependencies Scanned*: 49 (29 unique)
- *Vulnerable Dependencies*: 13
- *Vulnerabilities Found*: 160
- *Vulnerabilities Suppressed*: 0
- ...

Summary

Summary of Vulnerable Dependencies (click to show all)

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
hibernate-validator-6.0.18.Final.jar	cpe:2.3:a:hibernate:hibernate-validator:6.0.18:*:*:*:*:*:*:* cpe:2.3:a:redhat:hibernate_validator:6.0.18:*:*:*:*:*:*:*	pkg:maven/org.hibernate.validator/hibernate-validator@6.0.18.Final	MEDIUM	3	Highest	32
jackson-databind-2.10.2.jar	cpe:2.3:a:fasterxml:jackson-core:2.10.2:*:*:*:*:*:*:* cpe:2.3:a:fasterxml:jackson-databind:2.10.2:*:*:*:*:*:*:* cpe:2.3:a:fasterxml:jackson-modules-java8:2.10.2:*:*:*:*:*:*:*	pkg:maven/com.fasterxml.jackson.core/jackson-databind@2.10.2	HIGH	6	Highest	39
json-path-2.4.0.jar	cpe:2.3:a:json-path:jayway_jsonpath:2.4.0:*:*:*:*:*:*:*	pkg:maven/com.jayway.jsonpath/json-path@2.4.0	MEDIUM	1	Highest	33
json-smart-2.3.jar	cpe:2.3:a:json-smart_project:json-smart:2.3:*:*:*:*:*:*:* cpe:2.3:a:json-smart_project:json-smart-v2:2.3:*:*:*:*:*:*:*	pkg:maven/net.minidev/json-smart@2.3	HIGH	2	Highest	45
log4j-api-2.12.1.jar	cpe:2.3:a:apache:log4j:2.12.1:*:*:*:*:*:*:*	pkg:maven/org.apache.logging.log4j/log4j-api@2.12.1	MEDIUM	2	Highest	42
logback-core-1.2.3.jar	cpe:2.3:a:qos:logback:1.2.3:*:*:*:*:*:*:*	pkg:maven/ch.qos.logback/logback-core@1.2.3	HIGH	2	Highest	31
snakeyaml-1.25.jar	cpe:2.3:a:snakeyaml_project:snakeyaml:1.25:*:*:*:*:*:*:*	pkg:maven/org.yaml/snakeyaml@1.25	CRITICAL	8	Highest	44
spring-boot-2.2.4.RELEASE.jar	cpe:2.3:a:vmware:spring_boot:2.2.4:release:*:*:*:*:*:*	pkg:maven/org.springframework.boot/spring-boot@2.2.4.RELEASE	CRITICAL	3	Highest	39
spring-core-5.2.3.RELEASE.jar	cpe:2.3:a:pivotal_software:spring_framework:5.2.3:release:*:*:*:*:*:*:* cpe:2.3:a:springsource:spring_framework:5.2.3:release:*:*:*:*:*:*:* cpe:2.3:a:vmware:spring_framework:5.2.3:release:*:*:*:*:*:*:*	pkg:maven/org.springframework/spring-core@5.2.3.RELEASE	CRITICAL*	12	Highest	36
spring-hateoas-1.0.3.RELEASE.jar	cpe:2.3:a:vmware:spring_hateoas:1.0.3:release:*:*:*:*:*:*	pkg:maven/org.springframework.hateoas/spring-hateoas@1.0.3.RELEASE	MEDIUM	1	Highest	43
spring-web-5.2.3.RELEASE.jar	cpe:2.3:a:pivotal_software:spring_framework:5.2.3:release:*:*:*:*:*:*:* cpe:2.3:a:springsource:spring_framework:5.2.3:release:*:*:*:*:*:*:* cpe:2.3:a:vmware:spring_framework:5.2.3:release:*:*:*:*:*:*:*	pkg:maven/org.springframework/spring-web@5.2.3.RELEASE	CRITICAL*	13	Highest	34
tomcat-embed-core-9.0.30.jar	cpe:2.3:a:apache:tomcat:9.0.30:*:*:*:*:*:*:* cpe:2.3:a:apache_tomcat:apache_tomcat:9.0.30:*:*:*:*:*:*:*	pkg:maven/org.apache.tomcat.embed/tomcat-embed-core@9.0.30	CRITICAL*	53	Highest	30
tomcat-embed-websocket-9.0.30.jar	cpe:2.3:a:apache:tomcat:9.0.30:*:*:*:*:*:*:* cpe:2.3:a:apache_tomcat:apache_tomcat:9.0.30:*:*:*:*:*:*:*	pkg:maven/org.apache.tomcat.embed/tomcat-embed-websocket@9.0.30	CRITICAL*	54	Highest	30

* indicates the dependency has a known exploited vulnerability

Dependencies (vulnerable)

Original dependency report screenshot. The standalone full-resolution image in screenshots/dependency-check.png is easier to enlarge than this document preview.

The scan was not clean. No new libraries were added in the reported refactor, but that fact alone does not establish that no vulnerabilities were introduced. The retained report does not demonstrate remediation or production readiness.

References

[1] NIST. FIPS 197, Advanced Encryption Standard. https://csrc.nist.gov/pubs/fips/197/final

[2] Oracle. Java SE 11 MessageDigest. https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/security/MessageDigest.html

[3] Oracle. Java SE 11 keytool. https://docs.oracle.com/en/java/javase/11/tools/keytool.html

[4] OWASP. Transport Layer Security Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Security_Cheat_Sheet.html

[5] OWASP. Dependency-Check. https://owasp.org/projects/dependency-check